

Observabilidade LioConecta

Relatório de implementação e evidências dos testes E2E (Fases 1–8).
Inclui hub admin, ingestão de telemetria, APIs de consulta, correlação
ponta a ponta e suite automatizada backend/frontend.

Gerado em: 05/07/2026, 13:08:13

Branch: feature/observability-platform

Repositórios: LioConecta-FrontEnd + LioConecta.Backend

1. Resumo do que foi implementado

Fase	Entregável	Status
2 — Design	ADR, catálogo de eventos, LGPD, OTel, UI-SPEC, contratos API	Concluída
3 — Backend core	CorrelationMiddleware, GlobalExceptionHandler, AccessAudit, OTel	Concluída
4 — Frontend core	TelemetryClient, ErrorBoundary, session correlation, page views	Concluída
5 — Persistência	3 tabelas + ingestão batch + retention job	Concluída
6 — APIs admin	summary, errors, page-views, access-events, metrics, investigate	Concluída
7 — UI hub	/admin/observabilidade — 12 KPIs, 5 abas, timeline	Concluída
8 — Testes E2E	6 cenários backend + vitest + Playwright	Concluída

2. Arquitetura (visão operacional)

Frontend React

- └ TelemetryClient (batch 30s / 20 eventos, sendBeacon em pagehide)
- └ SessionCorrelationId + headers X-Correlation-Id / X-Session-Id
- └ ObservabilityHubPage (/admin/observabilidade)

Backend ASP.NET Core

- └ POST /api/v1/telemetry/events | page-views → observability_event, page_view
- └ AccessAuditMiddleware + auth handler → access_event
- └ Audit pipeline existente → audit_events
- └ GET /api/v1/admin/observability/* → consulta + investigate

Correlação: mesmo correlationId liga page view → request HTTP → access → audit → timeline admin

3. Cenários de teste backend (integração)

Arquivo: ObservabilityE2EScenariosTests.cs — **APROVADO**

Cenário 1 — Correlation ID em erros HTTP

Quando uma requisição falha, o backend propaga o mesmo UUID informado em X-Correlation-Id tanto no header de resposta quanto no corpo ProblemDetails (campo correlationId). Isso permite rastrear erros no hub e na trilha.

Teste: Scenario1_CorrelationId_PropagatesOnHttpError

Cenário 2 — Ingestão batch com redaction LGPD

O endpoint POST /api/v1/telemetry/events aceita lotes de eventos. Campos sensíveis (password, token) são removidos pelo TelemetryRedactor antes da persistência; apenas chaves da allowlist são gravadas em metadata.json.

Teste: Scenario2_TelemetryBatch_PersistsWithRedaction

Cenário 3 — Page views persistidos

Navegação SPA enviada via POST /api/v1/telemetry/page-views grava page_name, route_template, module, duração e correlationId na tabela page_views.

Teste: Scenario3_PageViewBatch_PersistsNavigationEvent

Cenário 4 — Access audit em rotas admin

GETs em rotas sensíveis (/api/v1/admin/**) geram access_events com event_name Resource.Viewed, correlacionados ao header X-Correlation-Id.

Teste: Scenario4_AdminGetAccessAudit_PersistsResourceViewed

Cenário 5 — APIs admin de consulta

Endpoints /admin/observability/summary, /errors e /metrics retornam KPIs, listas paginadas e séries temporais consumidas pelo hub React.

Teste: Scenario5_AdminQueryEndpoints_ReturnAggregatedData

Cenário 6 — Timeline unificada por correlationId

GET /admin/observability/investigate agrega page_views, access_events, observability_events e audit_events ordenados cronologicamente — cenário ponta a ponta do MD.

Teste: Scenario6_InvestigateTimeline_UnifiesAllSourcesByCorrelationId

Saída completa — dotnet test (Observability)

```
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\src\LioConecta.Api\LioConecta.Api.csproj : warning NU1902: O pacote 'OpenTelemetry.Exporter.OpenTelemetryProtocol' 1.9.0 tem uma moderado vulnerabilidade de gravidade conhecida, https://github.com/advisories/GHSA-4625-4j76-fww9
LioConecta.Domain ->
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\src\LioConecta.Domain\bin\Debug\net8.0\LioConecta.Domain.dll
LioConecta.Application ->
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\src\LioConecta.Application\bin\Debug\net8.0\LioConecta.Application.dll
LioConecta.Infrastructure ->
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\src\LioConecta.Infrastructure\bin\Debug\net8.0\LioConecta.Infrastructure.dll
LioConecta.Api ->
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\src\LioConecta.Api\bin\Debug\net8.0\LioConecta.Api.dll
LioConecta.IntegrationTests ->
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\tests\LioConecta.IntegrationTests\bin\Debug\net8.0\LioConecta.IntegrationTests.dll
Execução de teste para
C:\Users\leonardo.mendes\Projects\LioConecta.Backend\tests\LioConecta.IntegrationTests\bin\Debug\net8.0\LioConecta.IntegrationTests.dll (.NETCoreApp,Version=v8.0)
1 arquivos de teste no total corresponderam ao padrão especificado.

Aprovado! - Com falha: 0, Aprovado: 16, Ignorado: 0, Total: 16,
Duração: 3 s - LioConecta.IntegrationTests.dll (net8.0)
```

4. Testes unitários frontend (Vitest)

Arquivos: sessionCorrelation, telemetryClient, routeCatalog — **APROVADO**

- **sessionCorrelation** — IDs estáveis por sessão; refresh de correlation por navegação; headers aplicados.
- **telemetryClient** — flush automático ao atingir 20 eventos; falha de rede não derruba a app; telemetria desligada quando VITE_OBSERVABILITY_ENABLED=false.
- **routeCatalog** — rota /admin/observabilidade mapeada para módulo admin.

```
> lioconecta-frontend@1.0.0 test
> vitest run

RUN v3.2.6 C:/Users/leonardo.mendes/Projects/LioConecta-FrontEnd

✓ src/telemetry/sessionCorrelation.test.ts (3 tests) 8ms
✓ src/telemetry/routeCatalog.test.ts (1 test) 4ms
✓ src/telemetry/telemetryClient.test.ts (3 tests) 97ms

Test Files  3 passed (3)
   Tests   7 passed (7)
   Start at 13:07:52
   Duration 2.31s (transform 251ms, setup 106ms, collect 239ms, tests 109ms,
environment 4.33s, prepare 662ms)
```

5. Testes E2E UI (Playwright)

Arquivo: e2e/observability-hub.spec.ts — **APROVADO**

```
> lioconecta-frontend@1.0.0 test:e2e
> playwright test

Running 3 tests using 3 workers

ok 2 [chromium] › e2e\observability-hub.spec.ts:104:3 › Observability hub E2E ›
renders KPIs and errors tab for admin user (3.6s)
ok 3 [chromium] › e2e\observability-hub.spec.ts:114:3 › Observability hub E2E ›
investigate tab loads timeline for correlation id (3.9s)
ok 1 [chromium] › e2e\observability-hub.spec.ts:125:3 › Observability hub E2E ›
links tab exposes related admin destinations (4.0s)

3 passed (5.6s)

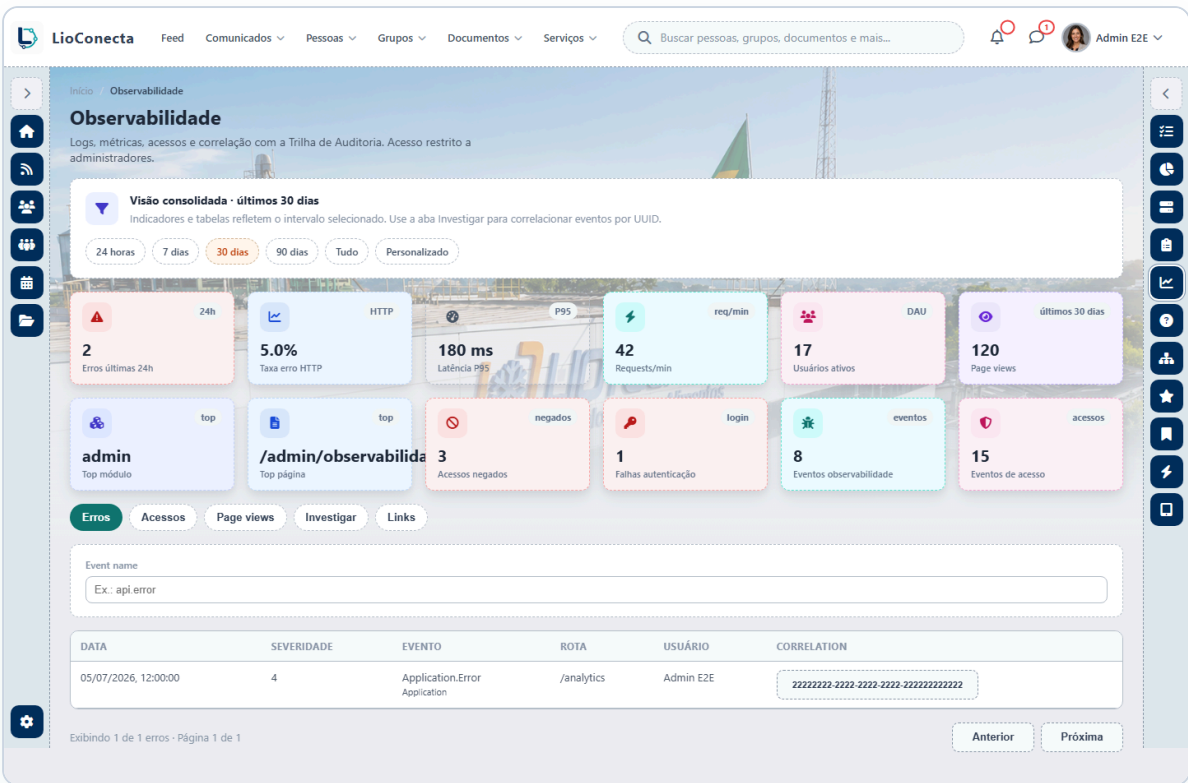
(node:23116) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env
being set.
(Use `node --trace-warnings ...` to show where the warning was created)
(node:28396) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env
being set.
(Use `node --trace-warnings ...` to show where the warning was created)
(node:33488) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env
being set.
(Use `node --trace-warnings ...` to show where the warning was created)
```

6. Evidências visuais — Hub de Observabilidade

Capturas realizadas com API mockada (mesmo padrão dos testes E2E), simulando respostas admin reais.

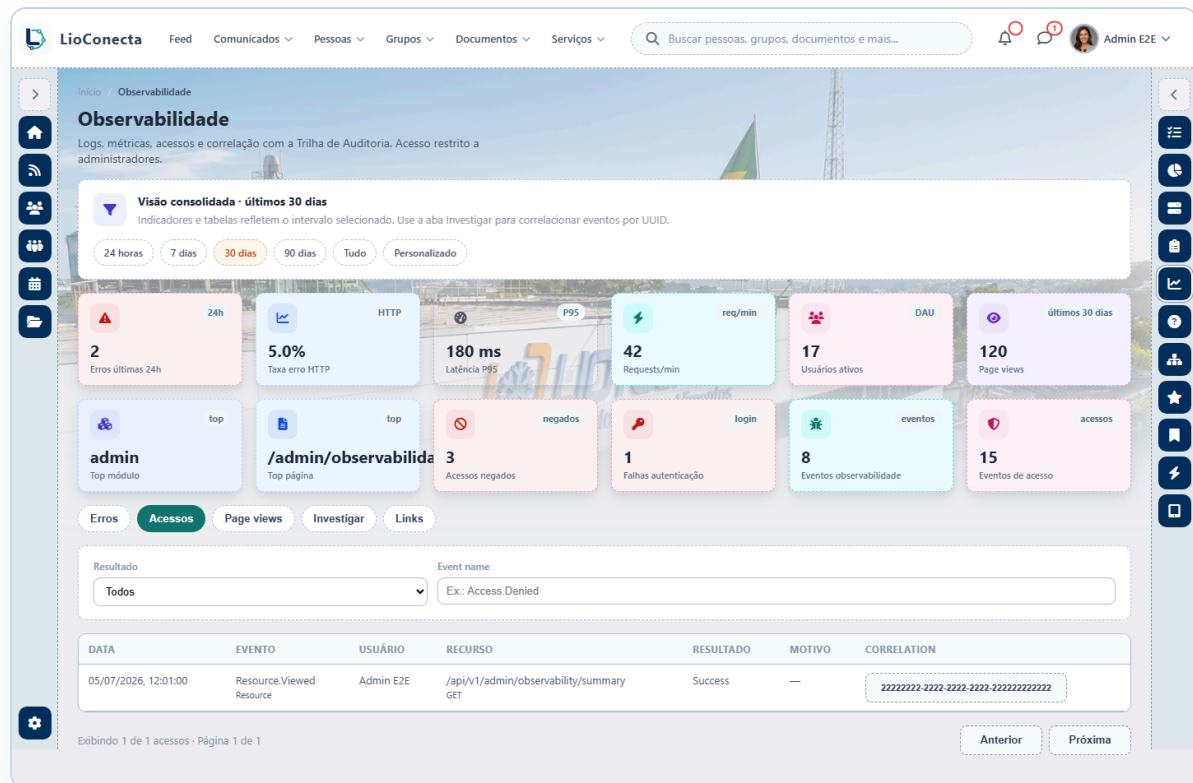
Hub — KPIs e aba Erros

Doze indicadores operacionais no topo; filtros de período reutilizam o padrão da Trilha de Auditoria. A aba Erros lista eventos $\text{severity} \geq \text{Error}$ com correlationId clicável.



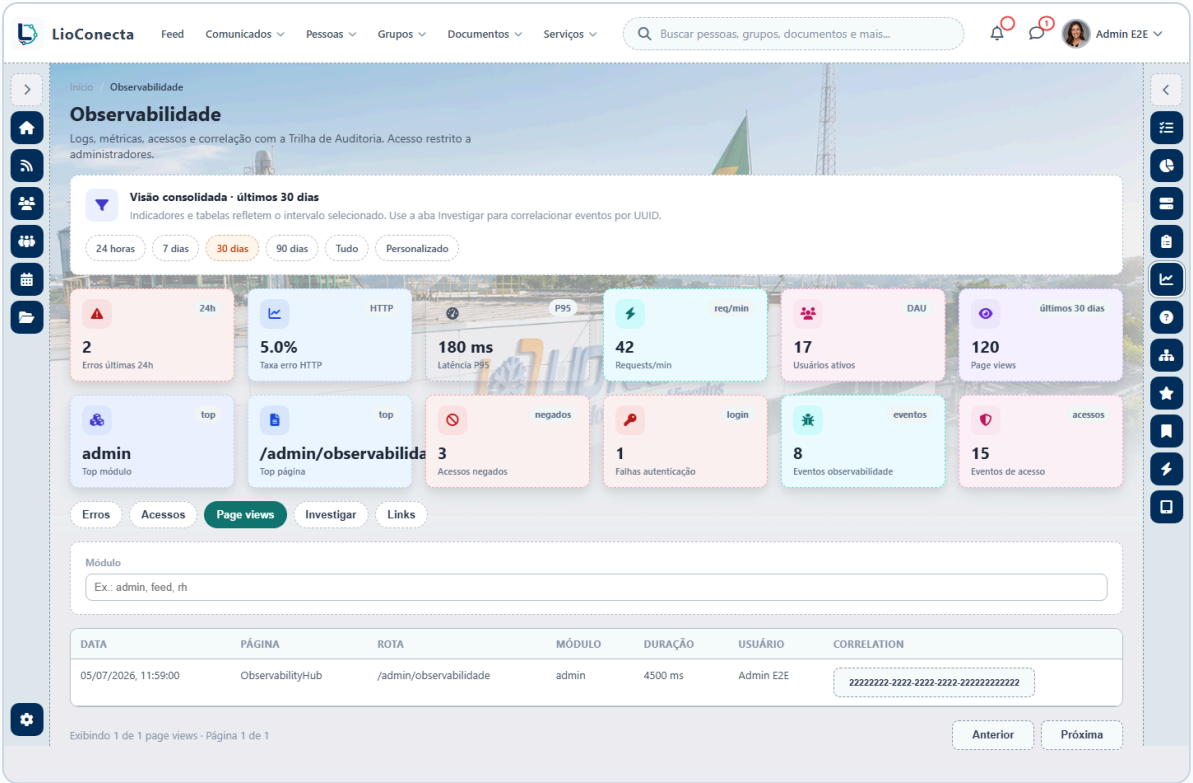
Hub — aba Acessos

Eventos de leitura/autorização (Resource.Viewed, Access.Denied etc.) com filtros por resultado e event_name.



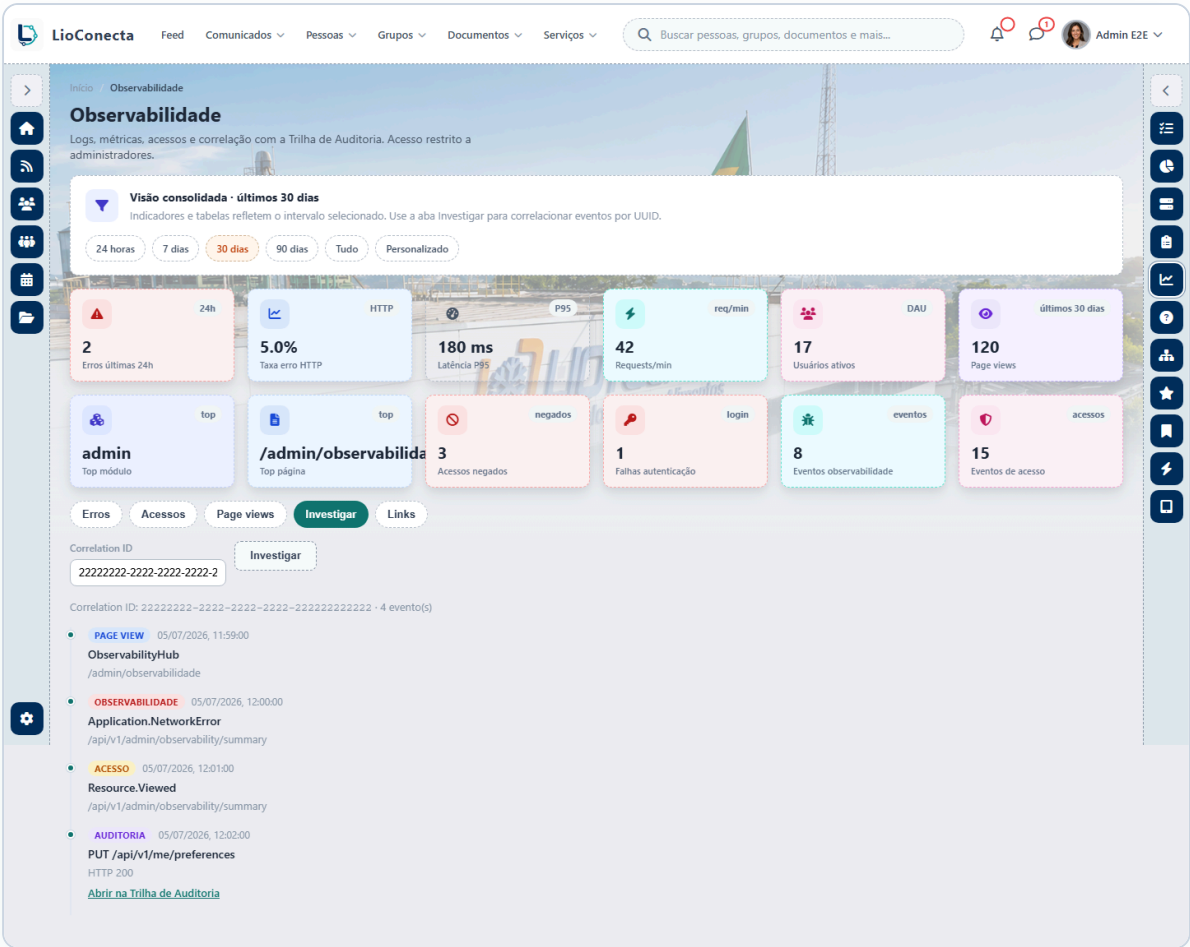
Hub — aba Page views

Histórico de navegação SPA: page_name, rota, módulo, duração e usuário.



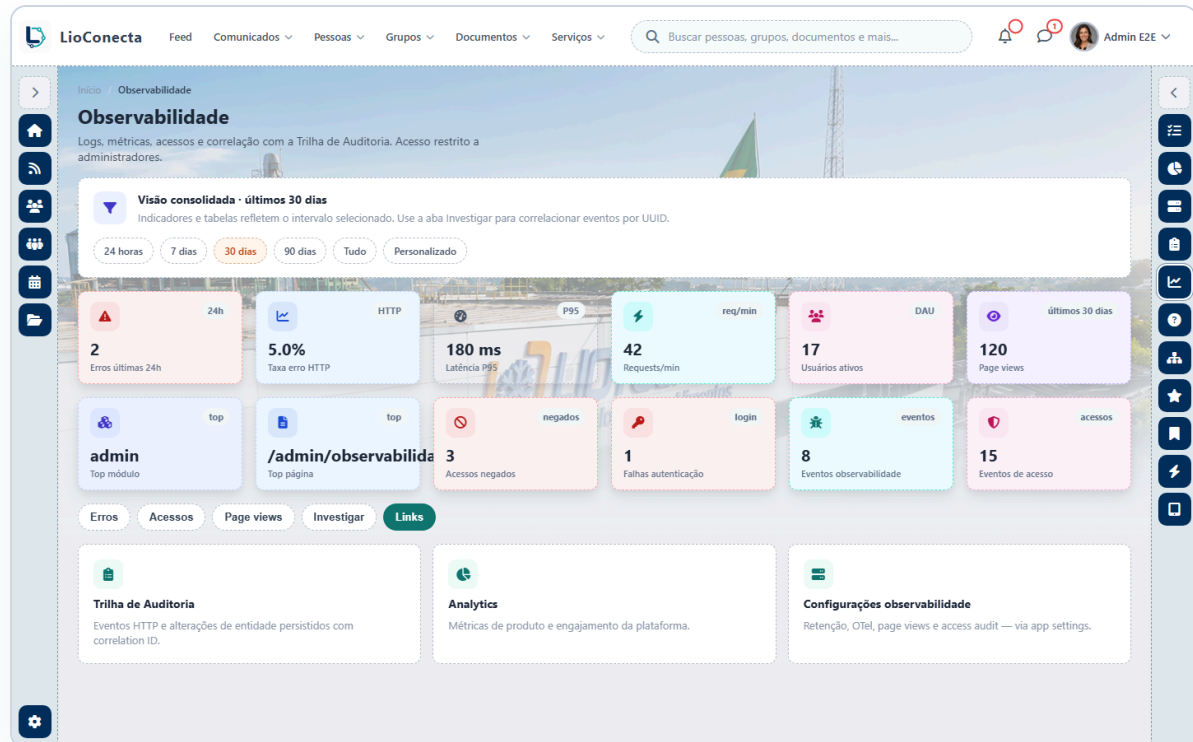
Hub — aba Investigar (timeline unificada)

Investigação por UUID: timeline vertical com badges por fonte (page view, ops, acesso, auditoria) e link para a Trilha.



Hub — aba Links relacionados

Atalhos para Trilha de Auditoria, Analytics e Configurações backend (categoria observability).



7. Como reproduzir

```
# Backend
dotnet test LioConecta.Backend/tests/LioConecta.IntegrationTests --filter
"Observability"

# Frontend
npm run test
npm run test:e2e

# Regenerar este PDF
node tools/generate-observability-e2e-report.mjs
```